

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA6502	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE-AI	Date:	20-08-2026

Instructions to Students

- Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S.Muskan**

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

10. FAQ-Based Generative AI Chatbot – Create an AI chatbot using a collection of at least 30 frequently asked questions and answers. The chatbot should understand direct and paraphrased questions and generate appropriate responses.

24. Summarizer with Length Control – Develop an AI summarization application that allows users to select summary length such as short, medium, or detailed. Demonstrate how the output changes for the same input text.

10.FAQ-Based Generative AI Chatbot

INTRODUCTION

Students frequently need quick answers to common college questions. Searching through notices, handbooks, or websites for simple information can be time-consuming. A FAQ chatbot can provide an interactive way to retrieve such information. This project develops a Python-based College FAQ Chatbot with a user-friendly Streamlit interface.

The application uses semantic similarity rather than relying only on exact keyword matching. This enables it to identify questions with similar meanings even when the wording is different. Generative AI can then be used to produce a concise answer from the selected FAQ.

PROBLEM STATEMENT

Develop an AI chatbot using a collection of at least 30 frequently asked questions and answers. The chatbot should understand direct and paraphrased student questions, provide appropriate responses, handle invalid or unrelated inputs, and present the functionality through a user-friendly interface.

OBJECTIVES

1. Create a college-specific FAQ knowledge base containing at least 30 FAQs.
2. Develop an interactive chatbot using Python and Streamlit.
3. Use semantic embeddings to identify the most relevant FAQ.
4. Support direct and paraphrased questions.
5. Use Gemini 2.5 Flash for natural-language answer generation when available.
6. Provide a fallback response for questions outside the knowledge base.
7. Test the application using multiple input cases.
8. Document limitations and possible future improvements.

SCOPE

The current system focuses on FAQ-based college information. The knowledge base can cover admissions, academics, examinations, library, hostel, scholarships, placements, and other frequently requested student services. The design can later be extended to larger document collections, multilingual interaction, RAG, administrative dashboards, and deployment as an online service.

TECHNOLOGIES AND TOOLS

Technology / Tool	Purpose
Python	Main programming language
Streamlit	Interactive web-based user interface
Sentence Transformers	Convert text into semantic embeddings
all-MiniLM-L6-v2	Lightweight sentence embedding model
NumPy	Similarity calculation and numerical processing
Gemini 2.5 Flash	Natural-language answer generation
python-dotenv	Load API key from .env
FAQ Dataset	College-specific knowledge base

SYSTEM REQUIREMENTS

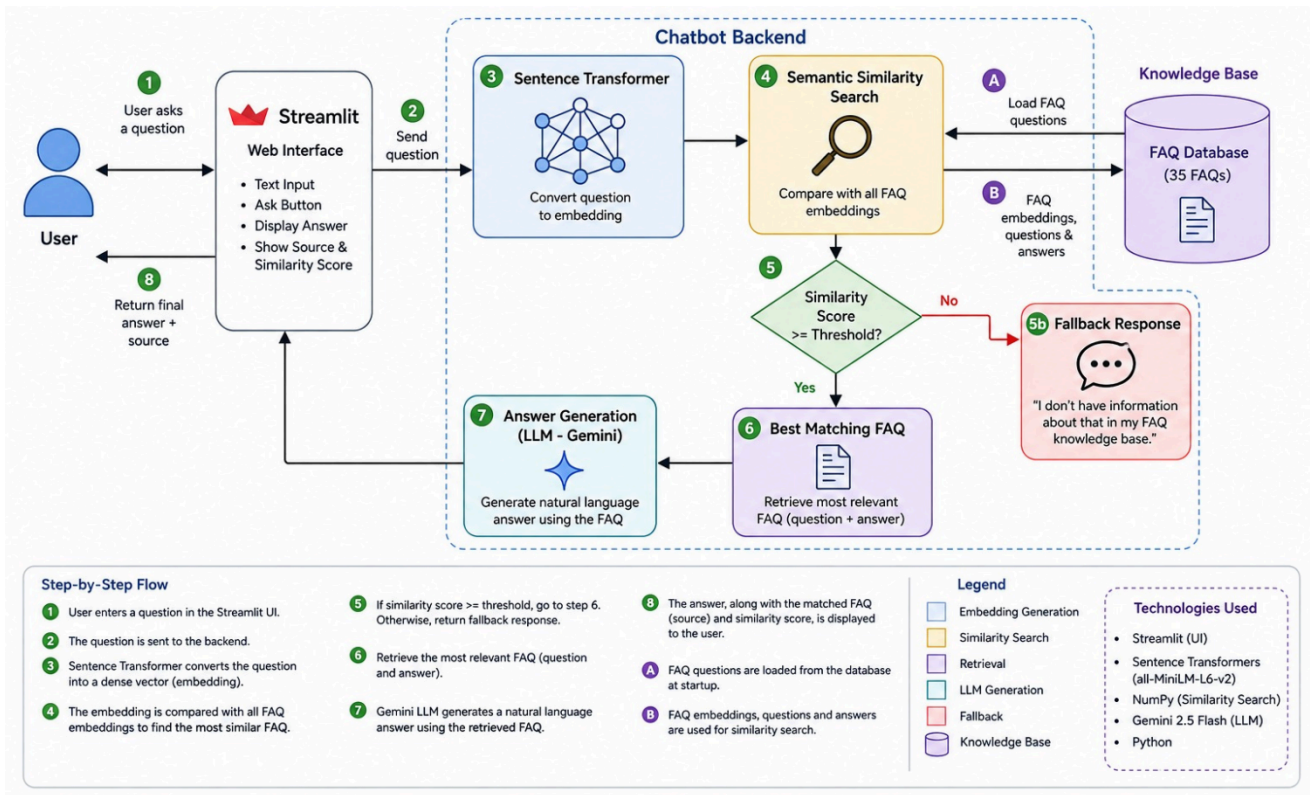
Hardware Requirements

- Computer/laptop with at least 4 GB RAM; 8 GB or more is recommended.
- Adequate storage for Python packages and the embedding model.
- Internet connection when using the Gemini API.

Software Requirements

- Windows / Linux / macOS
- Python 3.x
- Python IDLE or another Python IDE
- Streamlit
- Sentence Transformers
- NumPy
- python-dotenv
- Google GenAI SDK

SYSTEM ARCHITECTURE



Architecture Components

Component	Description
User	Student who enters a question.
Streamlit UI	Collects input and displays responses.
FAQ Knowledge Base	Stores 35 college-related question-answer pairs.
Sentence Transformer	Creates embeddings representing semantic meaning.
Similarity Search	Compares the query embedding with FAQ embeddings.
Threshold	Determines whether the retrieved FAQ is sufficiently relevant.
Gemini 2.5 Flash	Generates a concise answer from the retrieved FAQ.
Fallback	Handles unrelated or unsupported questions.

METHODOLOGY / WORKING PRINCIPLE

- 1. FAQ Knowledge Base:** The application stores a collection of 35 frequently asked questions and answers in `faq_data.py`.
- 2. Embedding Generation:** The all-MiniLM-L6-v2 Sentence Transformer converts FAQ questions into numerical embeddings.
- 3. User Query:** A student enters a question through the Streamlit interface.
- 4. Query Embedding:** The same embedding model converts the user question into a vector.
- 5. Similarity Calculation:** The query vector is compared with FAQ vectors using normalized vector dot products, representing cosine similarity.
- 6. FAQ Retrieval:** The FAQ with the highest similarity score is selected.
- 7. Threshold Check:** If the score is below the configured threshold, the chatbot returns a fallback response.
- 8. Generative Answer:** When a relevant FAQ is available, Gemini 2.5 Flash can generate a concise answer based only on the retrieved FAQ.
- 9. Display:** The result is displayed in the Streamlit interface.

ALGORITHM

Start the application.

Load the FAQ knowledge base.

Load the Sentence Transformer model.

Generate embeddings for the FAQ questions.

Accept a question from the student.

Generate an embedding for the student question.

Calculate similarity between the query and all FAQ embeddings.

Select the FAQ with the highest similarity.

Compare the score with the similarity threshold.

If the score is below the threshold, return the fallback response.

Otherwise, use the retrieved FAQ to generate the final answer using Gemini when API access is available.

Display the answer in the user interface.

Stop.

IMPLEMENTATION

Project Folder Structure

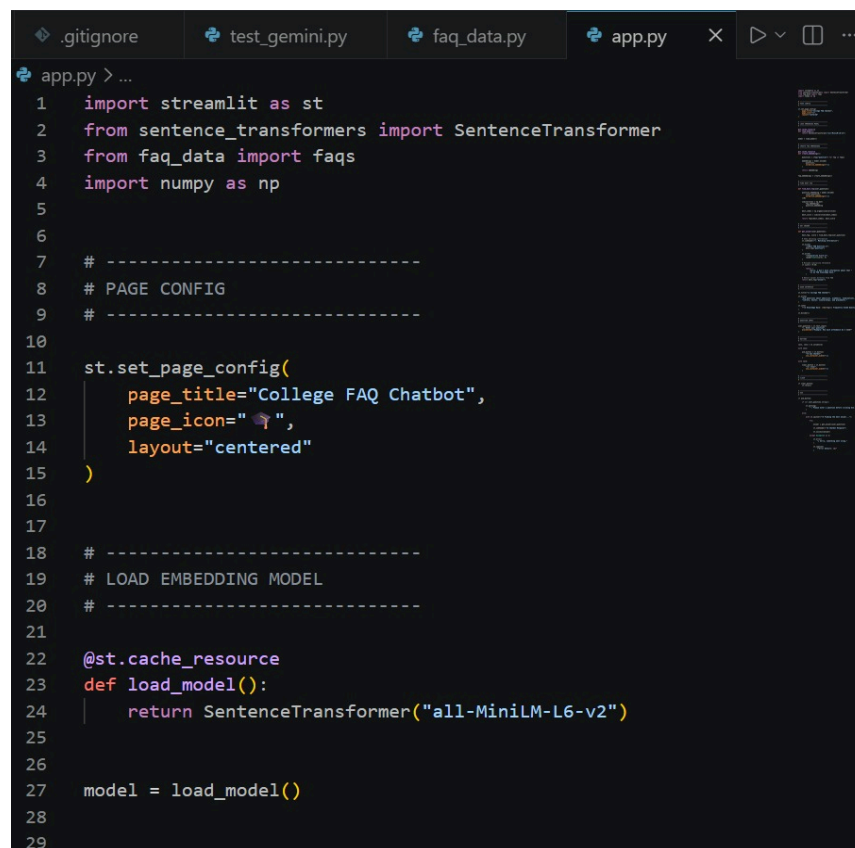
college_faq_chatbot/

```
|
|— app.py
|— faq_data.py
|— .env
|— requirements.txt
└— screenshots/
    |— 01_home_page.png
    |— 02_direct_question.png
    |— 03_paraphrased_question.png
    |— 04_unrelated_question.png
    └— 05_clear_button.png
```

Important Implementation Logic

The implementation contains separate logical sections for page configuration, loading the environment, model

loading, FAQ embedding creation, FAQ retrieval, answer generation, and the Streamlit user interface.



```
.gitignore test_gemini.py faq_data.py app.py
app.py > ...
1 import streamlit as st
2 from sentence_transformers import SentenceTransformer
3 from faq_data import faqs
4 import numpy as np
5
6
7 # -----
8 # PAGE CONFIG
9 # -----
10
11 st.set_page_config(
12     page_title="College FAQ Chatbot",
13     page_icon="🤖",
14     layout="centered"
15 )
16
17
18 # -----
19 # LOAD EMBEDDING MODEL
20 # -----
21
22 @st.cache_resource
23 def load_model():
24     return SentenceTransformer("all-MiniLM-L6-v2")
25
26
27 model = load_model()
28
29
```

SOURCE CODE


```
.gitignore test_gemini.py faq_data.py X app.py
faq_data.py > ...
1 faqs = [
2     {
3         "question": "How can I apply for admission?",
4         "answer": "Students can apply for admission through the c
5     },
6     {
7         "question": "What documents are required for admission?",
8         "answer": "Students generally need academic certificates,
9     },
10    {
11        "question": "What courses does the college offer?",
12        "answer": "The college offers undergraduate and postgraduate pro
13    },
14    {
15        "question": "How can I get information about course fees?",
16        "answer": "Course fee details can be obtained from the college
17    },
18    {
19        "question": "Is there an entrance examination for admission?",
20        "answer": "Some programs may require an entrance examination, wh
21    },
22    {
23        "question": "What are the college working hours?",
24        "answer": "The regular college working hours are from 9:00 AM to
25    },
26    {
27        "question": "What is the minimum attendance requirement?",
28        "answer": "Students are generally required to maintain at least
29    },
30    ]
```

FAQ DATA

USER INTERFACE

The application uses Streamlit to provide a simple interface for entering questions and viewing responses. Screenshots should be inserted below as evidence of the working application.

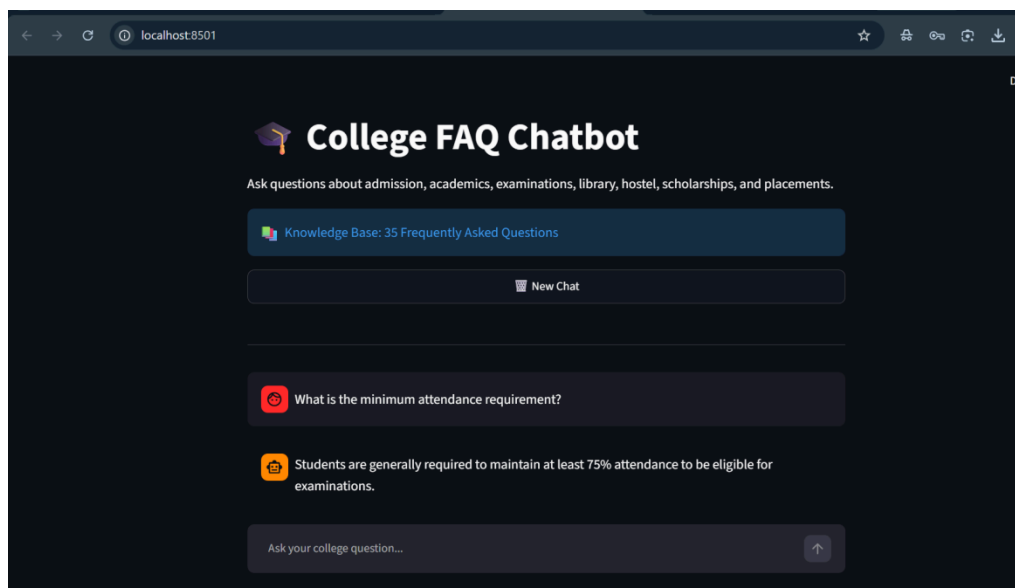


FIGURE 1 – HOME PAGE

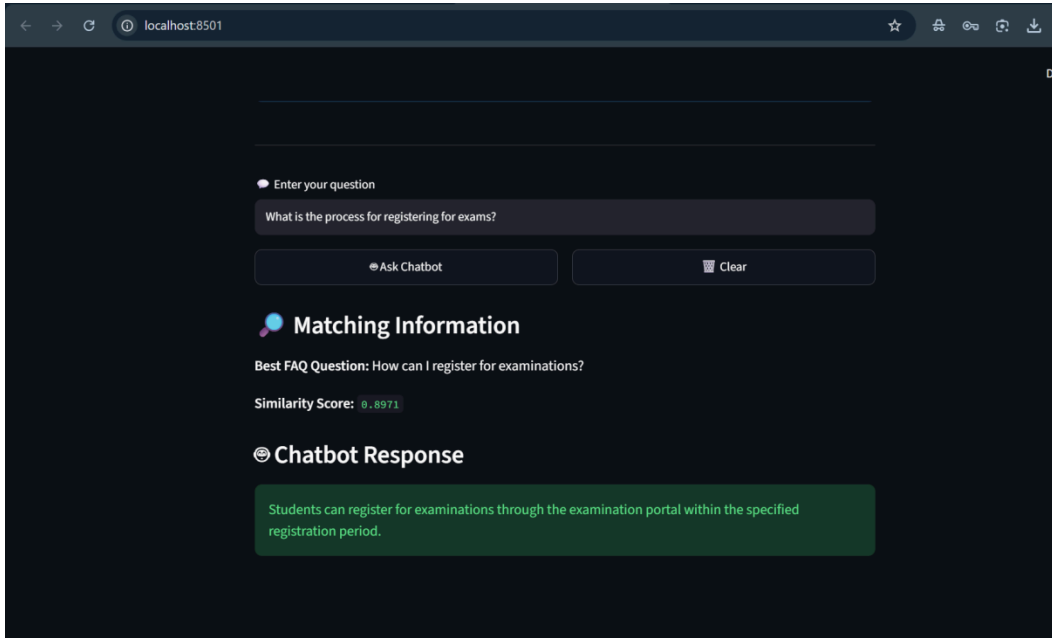


FIGURE 2 – DIRECT QUESTION

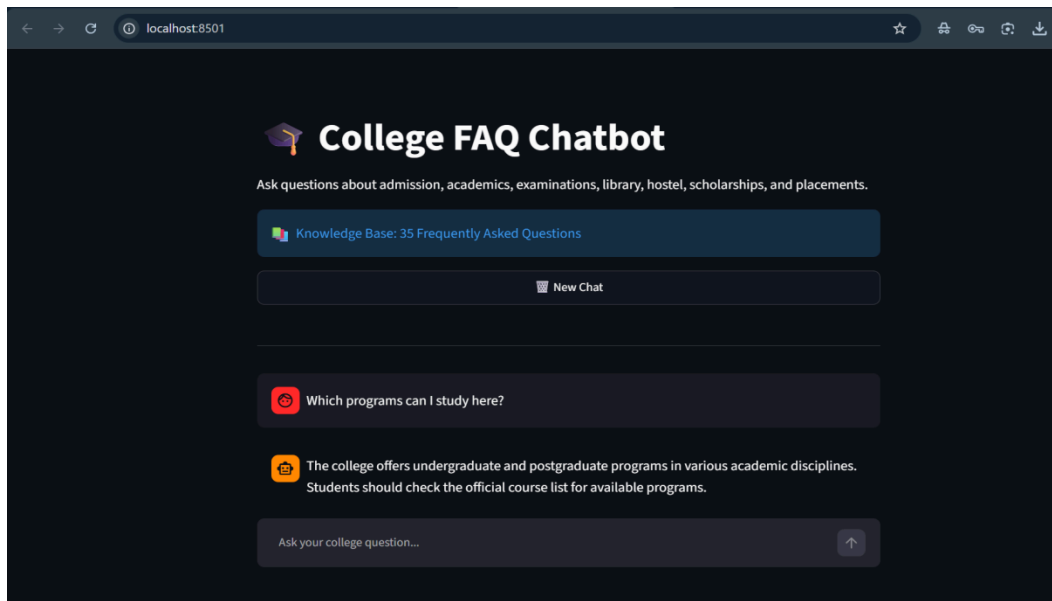


FIGURE 3 – PARAPHRASED QUESTION

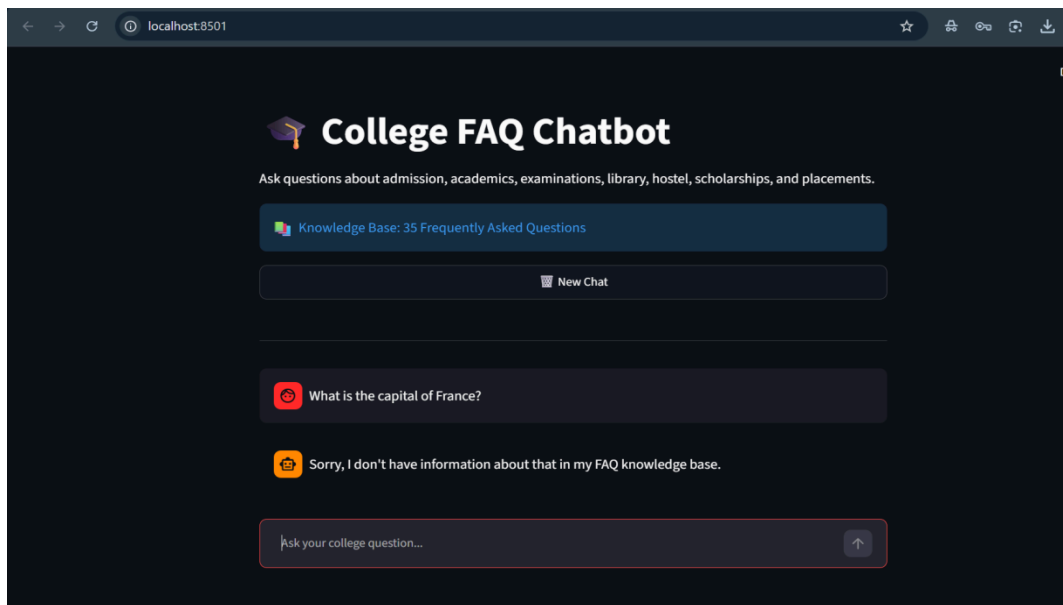


FIGURE 4 – UNRELATED QUESTION

TESTING AND TEST RESULTS

The application should be tested using direct questions, paraphrased questions, unrelated questions, and empty input. The following test cases provide a structured test plan.

ID	Type	Test Query	Expected Result	Actual Result	Status
T01	Direct	What is the minimum attendance requirement?	Attendance information	Attendance information	Pass
T02	Paraphrased	What percentage of classes should I attend?	Same attendance information	Same attendance information	Pass
T03	Paraphrased	How much attendance do	Attendance information	Attendance information	Pass

		I need for exams?			
T04	Direct	How can I apply for admission?	Admission procedure	Admission procedure	Pass
T05	Paraphrased	What do I need to do to join the college?	Admission information	Admission information	Pass
T06	Direct	What courses are offered by the college?	Course information	Course information	Pass
T07	Paraphrased	Which programs can I study here?	Course/program information	Course/program information	Pass
T08	Direct	How long can I borrow a library book?	Library borrowing information	Library borrowing information	Pass
T09	Paraphrased	For how many days can I keep a borrowed book?	Library borrowing information	Library borrowing information	Pass
T10	Direct	How do I register for examinations?	Exam registration information	Exam registration information	Pass
T11	Paraphrased	What is the process for registering for exams?	Exam registration information	Exam registration information	Pass
T12	Direct	Tell me about the college scholarship.	Scholarship information	Scholarship information	Pass

T13	Unrelated	What is the capital of France?	Fallback response	Fallback response	Pass
T14	Unrelated	What will the weather be tomorrow?	Fallback response	Fallback response	Pass
T15	Empty	[Leave input empty]	Warning to enter a question	Warning to enter a question	Pass

Test Summary

Test Category	Number of Tests	Status
Direct Questions	6	6
Paraphrased Questions	6	6
Unrelated Questions	2	2
Empty / Invalid Input	1	1
Total	15	15

MODEL AND TECHNIQUE JUSTIFICATION

Sentence Transformer – all-MiniLM-L6-v2

The Sentence Transformer model is used because the application must understand direct and paraphrased questions. It converts text into embeddings that capture semantic meaning. This allows questions with different wording but similar meaning to be matched.

Semantic Similarity

Normalized embeddings are compared using vector dot products. With normalized vectors, this represents cosine similarity. The FAQ with the highest score is treated as the best semantic match.

Gemini 2.5 Flash

Gemini 2.5 Flash is selected for natural-language answer generation. The retrieved FAQ is included in the prompt, and the model is instructed to use only the provided FAQ information. This helps keep generated answers aligned with the college knowledge base.

Streamlit

Streamlit is used because it provides a simple way to build an interactive web interface entirely in Python without requiring a separate frontend framework.

ADVANTAGES

1. Simple and user-friendly interface.
2. Understands paraphrased questions through semantic embeddings.
3. Fast retrieval for a small FAQ knowledge base.
4. Natural-language answer generation using Gemini.
5. Fallback mechanism for unrelated questions.
6. Easy to extend by adding new FAQs.
7. Useful as a prototype for a college student support system.

LIMITATIONS

- The chatbot is limited to information available in the FAQ knowledge base.
- Answer quality depends on the quality and completeness of the FAQ data.
- Semantic retrieval can occasionally select an incorrect FAQ.
- Gemini API access depends on network connectivity and quota/rate limits.
- The current system does not maintain long-term conversation memory.
- FAQ updates currently require editing the data source manually.
- The application is not a replacement for official college administrative communication.

FUTURE IMPROVEMENTS

- Expand the FAQ database with more college-specific information.
- Use FAISS or ChromaDB for larger vector collections.
- Retrieve and combine multiple relevant FAQs.
- Add conversation memory and follow-up question support.
- Add multilingual support for Indian languages.
- Add an administrator interface for managing FAQs.

- Add feedback buttons for answer quality.
- Add document/PDF-based retrieval using RAG.
- Improve API error and quota handling.

CONCLUSION

The College FAQ Chatbot demonstrates a practical application of semantic search and generative AI in an educational setting. The system uses Sentence Transformers to identify the most relevant FAQ for a student query and can use Gemini 2.5 Flash to generate a concise response based on the retrieved information. Streamlit provides an accessible interface for interaction. The application is capable of handling direct and paraphrased queries and provides fallback handling for unsupported questions. The project can be further extended into a larger RAG-based college assistant.

24.Summarizer with Length Control

Aim

To develop an AI-based text summarization application that allows users to select different summary lengths such as Short, Medium, and Detailed, and to demonstrate how the generated output changes for the same input text.

Objectives

Develop an AI-based summarization application.

Accept text from the user.

Provide Short, Medium, and Detailed options.

Generate summaries with different levels of detail.

Demonstrate different outputs for the same input.

Provide an easy-to-use Streamlit interface.

Introduction

Text summarization is an important Natural Language Processing (NLP) task that reduces long text into a shorter version while preserving important information. In this experiment, a Generative AI-based summarization application accepts user text and provides Short, Medium, and Detailed summaries. The same input can therefore produce outputs with different levels of detail.

Technologies Used

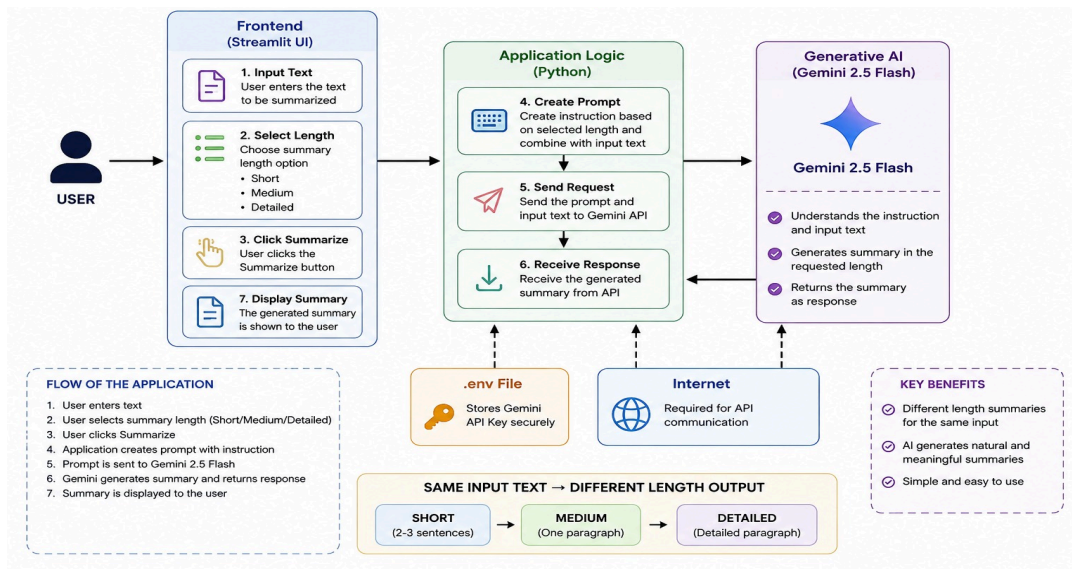
Technology	Purpose
Python	Main programming language
Streamlit	Web interface
Gemini 2.5 Flash	Summary generation
Google GenAI	Gemini API connection
python-dotenv	Secure API-key loading
.env	Stores API key

System Requirements

Hardware: Computer/Laptop, minimum 4 GB RAM, internet connection.

Software: Python 3.x, Streamlit, Google GenAI, python-dotenv, Gemini API key, web browser.

System Architecture



Working Principle

- Start the Streamlit application.
- Enter the text to be summarized.
- Select Short, Medium, or Detailed.
- Create an instruction based on the selected length.
- Send the text and instruction to Gemini 2.5 Flash.
- Receive the generated summary.
- Display the summary.

Algorithm

- Start.
- Import required libraries.
- Load Gemini API key from .env.
- Create Gemini client.
- Create Streamlit interface.
- Accept input text.
- Display summary-length options.

Create prompt according to selection.

Send prompt to Gemini.

Receive and display summary.

If input is empty, show warning.

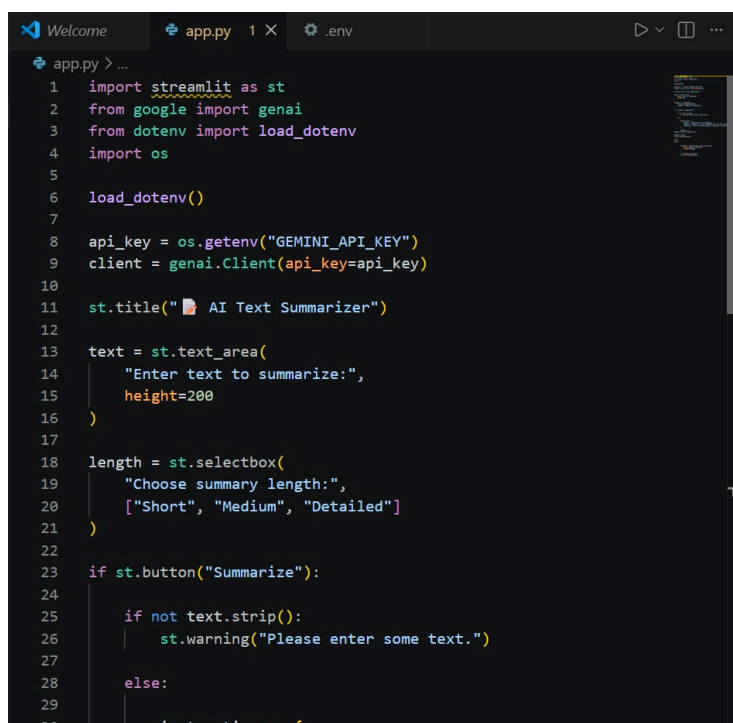
Stop.

Application Flow

*USER → STREAMLIT UI → ENTER TEXT → SELECT LENGTH → CREATE PROMPT →
GEMINI 2.5 FLASH → GENERATED SUMMARY → DISPLAY*

Program Code

Paste the final app.py code here, or replace the placeholder below with a source-code screenshot.

A screenshot of a code editor showing the source code for an application named 'app.py'. The code is written in Python and uses the Streamlit library for the user interface. It imports 'streamlit as st', 'google' for 'genai', and 'dotenv' for 'load_dotenv'. It also imports 'os'. The code sets the title of the application to 'AI Text Summarizer'. It creates a text area for the user to enter text to summarize, with a height of 200. It also creates a selectbox for the user to choose the summary length, with options 'Short', 'Medium', and 'Detailed'. A button labeled 'Summarize' is added. When the button is clicked, the code checks if the text is empty. If it is, it shows a warning message 'Please enter some text.'. If it is not empty, it proceeds to create a prompt based on the selected length and the entered text. The code is as follows:

```
1 import streamlit as st
2 from google import genai
3 from dotenv import load_dotenv
4 import os
5
6 load_dotenv()
7
8 api_key = os.getenv("GEMINI_API_KEY")
9 client = genai.Client(api_key=api_key)
10
11 st.title("📄 AI Text Summarizer")
12
13 text = st.text_area(
14     "Enter text to summarize:",
15     height=200
16 )
17
18 length = st.selectbox(
19     "Choose summary length:",
20     ["Short", "Medium", "Detailed"]
21 )
22
23 if st.button("Summarize"):
24     if not text.strip():
25         st.warning("Please enter some text.")
26     else:
27
28
29         instructions = f
```

Figure 2: app.py Source Code

Application Interface

The interface contains a text input area, summary-length selector, Summarize button, and output area.

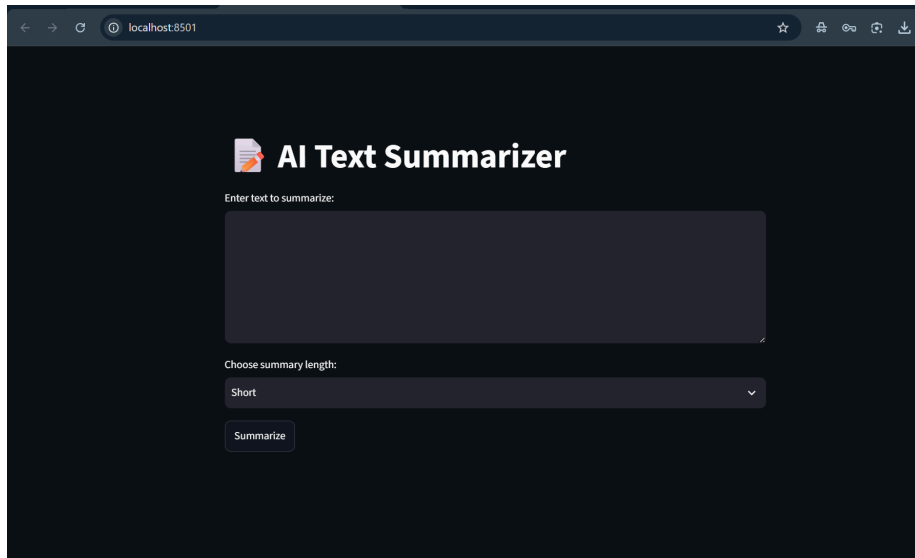


Figure 3: Initial Application Interface

Experimental Input

Artificial Intelligence is a technology that enables computers to perform tasks that normally require human intelligence. It can learn from data, recognize patterns, understand language, and make decisions. AI is widely used in healthcare, education, banking, transportation, and many other fields. Machine learning and deep learning are important technologies used to develop modern AI systems.

Test Case 1 — Short Summary

Use exactly the same input from Section 12.

Selected option: Short

Expected behavior: Brief summary containing the main idea.

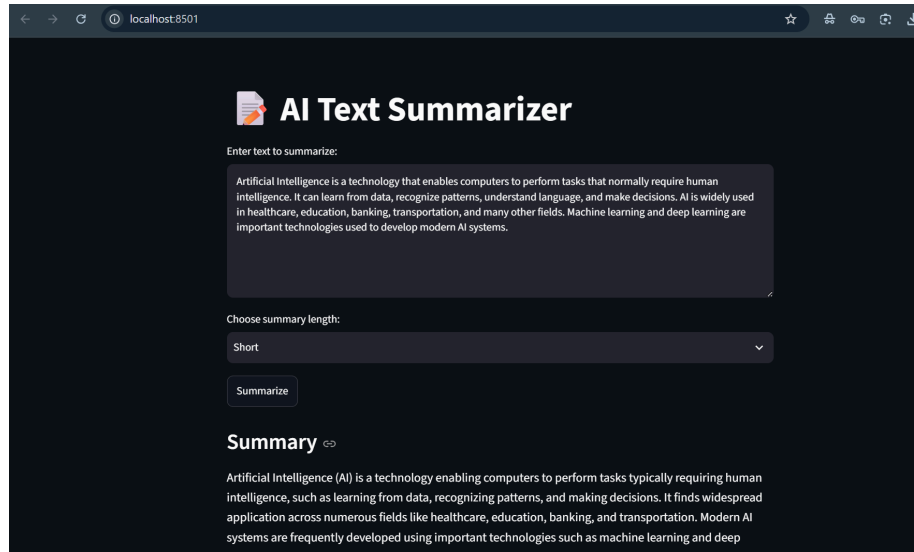


Figure: Short Summary Generated from the Same Input Text

Test Case 2 — Medium Summary

Use exactly the same input from Section 12.

Selected option: Medium

Expected behavior: Moderate summary containing the main points.

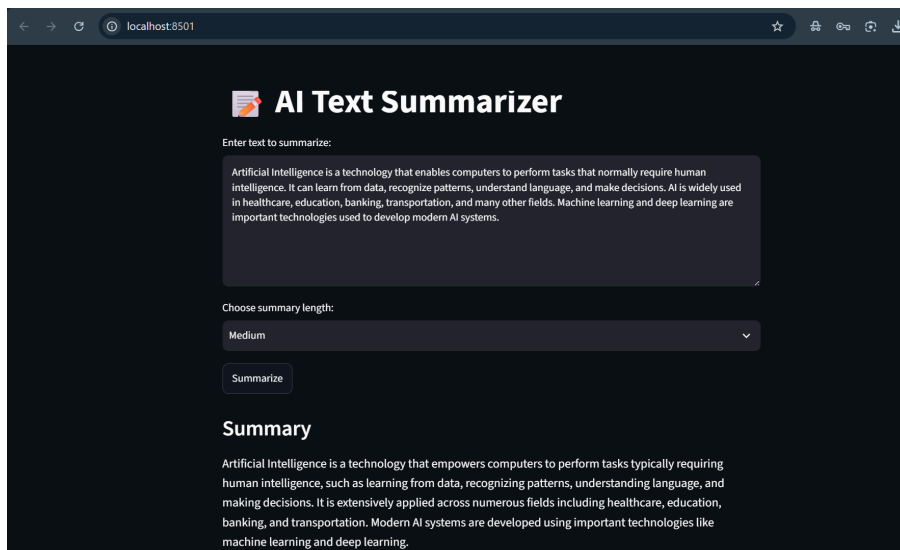


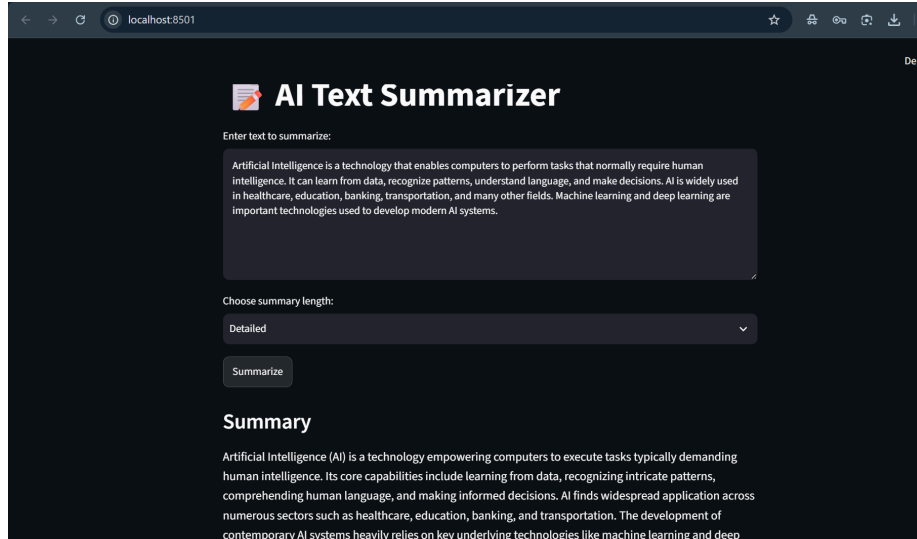
Figure: Medium Summary Generated from the Same Input Text

Test Case 3 — Detailed Summary

Use exactly the same input from Section 12.

Selected option: Detailed

Expected behavior: Longer summary containing more important information.



Comparison of Outputs

Feature	Short	Medium	Detailed
Input	Same	Same	Same
Length	Very short	Moderate	Longer
Information	Main idea	Main points	More details
Purpose	Quick	Normal	Deeper understanding

Test Cases

Test	Input	Selection	Expected Result	Status
TC01	AI paragraph	Short	Brief summary	Pass
TC02	Same paragraph	Medium	Moderate summary	Pass
TC03	Same paragraph	Detailed	Detailed summary	Pass
TC04	Empty	Any	Warning	Pass

Error Handling

If the user clicks Summarize without entering text, the application displays a warning and avoids an unnecessary AI request.

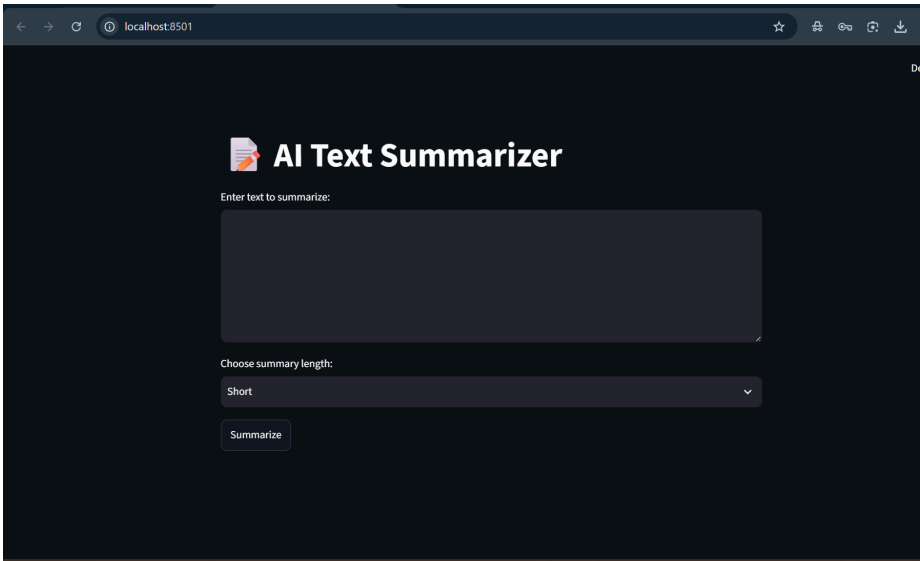


Figure 8: Empty Input Validation

Model Used

Gemini 2.5 Flash is used as the Generative AI model. The application sends the input together with an instruction specifying the desired summary length.

Examples: Short — summarize in 2–3 sentences. Medium — summarize in one paragraph with the main points.

Detailed — give a detailed summary covering all important points.

Advantages

- Simple interface
- Multiple summary lengths
- Natural-language AI summaries
- Same input produces different detail levels
- Easy Python implementation
- Can be extended to documents

Limitations

- Requires internet/API access
- API quotas may apply
- AI may occasionally omit details
- Current version uses manually entered text
- Large inputs may need additional handling

Future Improvements

- PDF/Word/TXT upload
- Multiple languages
- Custom summary length
- Word-count control
- Download summaries
- Summary history
- Voice input
- Online deployment

Result

The AI-based Text Summarizer was successfully developed using Python, Streamlit, and Gemini 2.5 Flash. It

generated Short, Medium, and Detailed summaries and demonstrated different output lengths for the same input text.

Conclusion

The Summarizer with Length Control demonstrates how Generative AI can be instructed to produce different levels of detail. The application combines Python, Streamlit, and Gemini 2.5 Flash to provide a practical and interactive summarization system.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						